

Chip Mullins

MSCS-532-B01

March 29, 2026

Assignment 5: Quicksort Algorithm Implementation, Analysis and Randomization

Introduction

In computer science, sorting algorithms are essential because they underlie efficient data processing. A particularly popular sorting algorithm is Quicksort. Due to the critical role randomization can play in algorithmic efficiency, examining both deterministic and randomized Quicksort implementations is valuable. By analyzing these two versions, we can understand the impact of randomization on Quicksort's performance and demonstrate the advantages of using a randomized approach.

Quicksort Algorithm Overview

Quicksort is a divide-and-conquer algorithm that sorts an array by first dividing it into two subarrays, then sorting each subarray, and finally combining the results. The division step involves selecting a pivot element from the array, then partitioning the array into two subarrays: one with elements less than or equal to the pivot, and one with elements greater than the pivot. Recursion is used to perform the sorting of subarrays, allowing Quicksort to efficiently handle large datasets.

Deterministic Quicksort Implementation

Below is a simple deterministic implementation of Quicksort in Python, where the first element is chosen as the pivot:

```
def quicksort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[0]  
    less = [x for x in arr[1:] if x <= pivot]  
    greater = [x for x in arr[1:] if x > pivot]  
  
    return quicksort(less) + [pivot] + quicksort(greater)
```

This approach is straightforward but can lead to poor performance on already sorted or nearly sorted input data.

Randomized Quicksort Implementation

To improve performance on various input data, the randomized version of Quicksort selects a random element as the pivot:

```
import random  
  
def randomized_quicksort(arr):  
    if len(arr) <= 1:  
        return arr
```

```
pivot = random.choice(arr)

arr_copy = arr.copy()

arr_copy.remove(pivot)

less = [x for x in arr_copy if x <= pivot]

greater = [x for x in arr_copy if x > pivot]

return randomized_quicksort(less) + [pivot] + randomized_quicksort(greater)
```

By randomly selecting pivots, this algorithm avoids worst-case performance scenarios caused by adversarially ordered inputs.

Time Complexity Analysis

The time complexity of Quicksort depends on how well the chosen pivots divide the array. It can be categorized as follows:

- Best Case: $O(n \log n)$, when pivots consistently divide arrays into balanced partitions.
- Average Case: $O(n \log n)$, with randomized pivot selection ensuring balanced partitions on average.
- Worst Case: $O(n^2)$, which occurs when pivots lead to highly unbalanced partitions, typical in deterministic Quicksort on sorted data.

Space Complexity

The space complexity of Quicksort is $O(\log n)$ due to recursion stack usage in average cases, which can grow to $O(n)$ in worst cases. Additionally, the current implementation uses additional memory for subarrays, introducing extra space overhead.

Impact of Randomization

Randomization in pivot selection helps avoid worst-case scenarios by ensuring that pivots are chosen independently of input data order. This maintains the expected time complexity of $O(n \log n)$, with worst-case performance being extremely rare.

Empirical Analysis

By testing both deterministic and randomized Quicksort variants on different types of input data, including sorted, nearly sorted, and random arrays, we observe the following:

Results Summary

- Deterministic Quicksort shows significantly slower performance on sorted input.
- Randomized Quicksort avoids performance degradation on sorted input.
- Randomized Quicksort consistently outperforms deterministic Quicksort across all input types.

Conclusion

Quicksort is an efficient sorting algorithm but is prone to poor performance in certain scenarios. The deterministic implementation is susceptible to worst-case performance on sorted inputs, while the randomized algorithm demonstrates better consistency and superior average-case performance. Due to its robust performance across diverse datasets, randomized Quicksort is

better suited for practical applications. Although the deterministic algorithm has educational value, only randomized Quicksort should be considered for real-world use.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1), 10–16.